A direct functional interpretation of arithmetic R in PTS-rk

October 20, 2026

Abstract

This development interprets the original functional presentation of Tarski–Mostowski–Robinson arithmetic R in the unchanged, annotated rank-based pure type system PTS-rk. Arithmetic expressions have a fixed Church representation with zero, successor, addition and multiplication. Certified observations supply equality and order, finite observations separate numerals, and one positive bounded-case invariant establishes the quantified axiom schemes. The formalization proves preservation of an independently defined classical first-order deduction system, complete proof/refutation results for ground atoms, and failure of faithfulness for arbitrary sentences. Constructive enumeration of finite typing derivations also gives executable interfaces returning actual annotated proof terms with proofs of their original PTS typing judgments.

1 The result and its scope

The library is `Interpretation_of_R.functional`. Its public entry point is `DirectRTheorem7.v`, and its principal declaration is `Theorem7.theorem7`. This is the result labelled `thm:direct-R` in the harmonized arithmetic manuscript, whose printed numbering gives Theorem 7. The construction is independent of the relational interpretation and of the set-model libraries. It shares structural lemmas, negative logic, finite-selector infrastructure and certificate search with the parent `Interpretation_of_R` library.

Write $A^\dagger$ for `FirstOrder.sentence_translation A`, and $\vdash p : A$ for the original `Typing.typ` judgment with `Ranks.RankSorts`, natural-number sorts and the empty context. Every proof term below is an element of `PTSrk.Syntax.Term nat 0`; open constructions use its scope-indexed counterpart. The abbreviation `Realizes A` means that such a term exists. It is not a new typing system or an assumed realization relation.

Theorem 1.1. *The declaration `Theorem7.theorem7` proves the conjunction of four claims.*

1. *Every sentence recognized by `FunctionalSyntax.axiom` has a closed originally typed inhabitant of its translation.*
2. *If `FunctionalSyntax.derives [] A`, then* $\exists p.\ \vdash p : A^\dagger$.
3. *For every ground equality or order atom, its numerical truth gives an inhabitant of its translation, and its numerical falsity gives an inhabitant of the translated atom's negation.*
4. *Some sentence has an inhabited translation but is not derivable in the independent source deduction system.*

The last clause is intentional: this is a deduction-preserving interpretation with exact ground proof/refutation behavior, not a conservativity theorem for all first-order sentences. Internal refutation means an actual term of type $A^\dagger \to \bot$. External uninhabitation of $\bot$ is a separate metatheoretic question and is not a conjunct of `theorem7`.

2 The unchanged target calculus and logical encoding

PTSrk has sorts U_i indexed by external natural numbers, with $U_i : U_0$ and product rule

$$\frac{\Gamma \vdash A : U_a \qquad \Gamma, x : A \vdash B : U_b}{\Gamma \vdash \Pi x : A.B : U_{\max(a+1,b)}}.$$

Abstractions retain their domain annotations. Conversion is the original beta conversion; neither cumulativity nor eta conversion is used. Ranks in this document are exact sort indices of displayed types, not implicit universe inclusions.

The proof encodings use the fixed answer type

$$\bot := \Pi Z : U_0.Z : U_1, \qquad \top := U_0 \to U_0 : U_1.$$

Set $\neg_c A = A \to \bot$ and $A \wedge_c B = (A \to B \to \bot) \to \bot$. A flat finite disjunction is

$$\bigvee_c (A_0, \ldots, A_k) := (A_0 \to \bot) \to \cdots \to (A_k \to \bot) \to \bot.$$

A proof type A is stable when $(\neg_c \neg_c A) \to A$ is inhabited. Negative atoms and the translated connectives are stable, and dependent products preserve stability of their codomain. This supports the classical rules needed by the source calculus while restricting elimination of negative encodings to stable targets. Positive domain evidence requires the separate construction described below.

Exact-rank lifts handle motives whose natural rank is too small. With $J_1 = \top$, $J_{k+1} = J_k \to J_k$, and canonical inhabitants j_k, the type $L_k(A) = J_{k-1} \to A$ has rank k whenever $A : U_a$, $a \leq k$ and $k \geq 2$. Abstraction and evaluation at j_{k-1} give maps between A and its lift; no cumulativity rule is introduced.

3 Arithmetic presentations and the admitted domain

The carrier in `Codes.Domain` is

$$D := \Pi X : U_1.\ X \to (X \to X) \to (X \to X \to X) \to (X \to X \to X) \to X : U_3.$$

For $\xi = (X, z, s, a, m)$ write $\mathrm{ev}_\xi(d) = d\,X\,z\,s\,a\,m$. The notation $\lambda\xi$ abbreviates five annotated abstractions, not a record constructor. The four closed operations are

$$\begin{aligned} 0_D &= \lambda\xi.z, & S_D d &= \lambda\xi.s(\mathrm{ev}_\xi(d)), \\ d +_D e &= \lambda\xi.a(\mathrm{ev}_\xi(d))(\mathrm{ev}_\xi(e)), & d \cdot_D e &= \lambda\xi.m(\mathrm{ev}_\xi(d))(\mathrm{ev}_\xi(e)). \end{aligned}$$

Thus addition and multiplication construct arithmetic expression trees. Their ground arithmetic equations will hold in interpreted equality; they are not beta computations to canonical numerals. Put $\nu_n = S_D^n 0_D$ for the canonical presentation of external n.

For $Q : D \to U_{10}$, let $\mathsf{Cl}_S(Q)$ express successor closure and $\mathsf{Cl}_f(Q)$ express binary closure under f. Define

$$\mathsf{Good}(d) := \Pi Q : D \to U_{10}.\ Q(0_D) \to \mathsf{Cl}_S(Q) \to \mathsf{Cl}_{+_D}(Q) \to \mathsf{Cl}_{\cdot_D}(Q) \to Q(d).$$

This rank-12 predicate is the fixed admitted domain. Its constructor certificates apply the corresponding closure argument to the children's certificates. There is no claim of induction

for arbitrary raw inhabitants of D, nor is the desired bounded-case theorem inserted into the domain definition. The rank-10 motive is fixed independently of every axiom index.

`DirectRCodes.v` constructs these expressions using the shared named assembler, then closes them into original de Bruijn terms. Assembly proves that names are resolved, not that terms are well typed. `DirectRFoundations.v` separately proves their original typing, often by the sound native checker. `FunctionalTerms.term_typed` and `FunctionalTerms.term_admitted` extend typing and admission compositionally to all source arithmetic terms under a typed, admitted environment.

4 The certified observation telescope

An observation has eight data parameters:

$$X : U_1, \quad z : X, \quad s : X \to X, \quad a, m : X \to X \to X, \quad \delta : X \to U_4, \quad R_0, C_0 : X \to X \to U_1.$$

Its local proof relations are $u \simeq v = R_0(u, v) \to \bot$ and $u \preceq v = C_0(u, v) \to \bot$. The data are followed by 23 separate proof parameters. Every object variable in a law is followed immediately by its δ guard. The certificates comprise:

- four domain laws: zero and closure under the three operations;
- equivalence of $\simeq$, congruence for successor, addition and multiplication, and congruence of $\preceq$ in both arguments;
- reflexivity and transitivity of $\preceq$;
- five arithmetic equations and five order propagation laws below.

With $1_X = sz$, the arithmetic and propagation laws are

$$\begin{aligned} a(z,u) &\simeq u, \quad a(su,v) \simeq s(a(u,v)),\\ m(z,u) &\simeq z, \quad m(su,v) \simeq a(v, m(u,v)),\\ m(u,z) &\simeq z, \end{aligned}$$

$$\begin{gathered} u \preceq su, \qquad u \preceq a(u,v), \qquad v \preceq a(u,v),\\ 1_X \preceq v \;\longrightarrow\; u \preceq m(u,v), \qquad 1_X \preceq u \;\longrightarrow\; v \preceq m(u,v). \end{gathered}$$

All certificates have rank at most five. In `Codes.observation_fields` they are literal successive binders, so a universal observation proof must accept every one of them. They are parameters of a definable proof type, not axioms added to PTS-rk.

Writing $\Pi\mathcal{A}$ for this complete telescope, define

$$\begin{aligned} E(d,e) &:= \Pi\mathcal{A}.\ \mathrm{ev}_{\mathcal{A}}(d) \simeq \mathrm{ev}_{\mathcal{A}}(e),\\ O(d,e) &:= \Pi\mathcal{A}.\ \mathrm{ev}_{\mathcal{A}}(d) \preceq \mathrm{ev}_{\mathcal{A}}(e),\\ G(d) &:= \Pi\mathcal{A}.\ \delta(\mathrm{ev}_{\mathcal{A}}(d)). \end{aligned}$$

These are `Codes.Equality`, `Codes.Order` and `Codes.G`, all with rank-six values. E and O are stable; G contains positive domain evidence. Fold equations prove constructor closure of G. Instantiating Good with the rank-10 lift of G gives $\mathsf{Good}(d) \to G(d)$. Opening the telescope and supplying these guards transfers its equations, congruences and preorder laws to observed arithmetic presentations.

External induction on m, n now produces originally typed ground addition and multiplication proofs, exported by `Kernel.R1_realized` and `Kernel.R2_realized`. This induction constructs target terms; native arithmetic automation only settles their numerical side conditions.

5 Finite observations and internal separation

For an external cutoff K, put $c = K + 1$ and use the chain $\{0, \dots, c\}$ with capped operations

$$s(i) = \min(i + 1, c), \quad a(i, j) = \min(i + j, c), \quad m(i, j) = \min(ij, c).$$

`DirectRArithmetic.v` establishes the numerical table properties. Their target implementation uses $q = K + 2$ rank-one selectors

$$F_q := \underbrace{U_1 \to \cdots \to U_1}_{q \text{ arguments}} \to U_1, \qquad e_i = \lambda Z_0 \cdots Z_{q-1}.Z_i.$$

A unary table routes a selector to its chosen output; a binary table uses a selector for its row followed by a selector for its column. For relation tables, a true entry returns $\bot$ and a false entry returns $\top$. Negating the result gives an inhabited relation in the true case and a refutable relation in the false case.

The selector-domain certificate is

$$\delta_q(u) := \Pi Q : F_q \to U_3.\ Q(e_0) \to \cdots \to Q(e_{q-1}) \to Q(u) : U_4.$$

Its elimination rule reduces a guarded law to the finitely many selector cases. This certifies every observation field, including binary closure, transitivity, congruence and the multiplication laws. `FiniteSpecialization.contextual_source_observation` assembles the full observation; `FunctionalEvaluation.finite_numeral_evaluation` relates numeral folds to their selectors. No assertion that every raw inhabitant of F_q is a selector is needed.

Theorem 5.1. *`FunctionalSeparation.no_back_realized` constructs, for every n, a closed proof of*

$$O(\nu_{n+1}, \nu_n) \to \bot.$$

More generally, reversed numeral comparisons and distinct-numeral equalities have internal refutations.

Proof. Choose cutoff $K = n$. The relevant folds are the distinct selectors e_{n+1} and e_n. Apply a proposed global order proof to the entire closed finite observation. Its conclusion reduces to $\top \to \bot$; application to the identity inhabitant of $\top$ yields $\bot$. Ground order, transitivity and equality congruence extend this argument to arbitrary reversed pairs and unequal numerals. □

The finite observations are separating target constructions. A single finite chain is not asserted to be a model of all source numeral-separation axioms, and numerical table validation alone is not the target proof.

6 The positive bounded-case invariant

For a fixed external n, set

$$\mathsf{Case}_n(d) := \bigvee_c \big(E(d,\nu_0),\ldots,E(d,\nu_n),O(\nu_{n+1},d)\big).$$

The last alternative is a global lower bound on d, rather than only overflow in one observation. This distinction makes the case lemma strong enough for both quantified arithmetic schemes.

Theorem 6.1. *`FunctionalCases.bounded_case_realized` proves, for every n,*

$$\Pi d : D.\ \mathsf{Good}(d) \to \mathsf{Case}_n(d).$$

Proof. Use the rank-10 positive continuation package

$$\mathsf{Inv}_n(d) := \Pi Z : U_8.\ (G(d) \to \mathsf{Case}_n(d) \to Z) \to Z.$$

Prove closure under all four constructors. Open the parent's continuation, then eliminate each child invariant at that same Z. This supplies positive G evidence and the negative case proof separately. Construct the parent's G evidence using the domain closure laws. To construct its case proof, first open the case continuations, so all remaining branches return $\bot$.

Zero takes the reflexive equality branch. Successor sends $i < n$ to $i+1$; the $i = n$ branch becomes the large branch. A large child remains large by $O(d, S_D d)$. Addition combines two finite cases by their ground sum; if that sum exceeds n, ground comparison supplies the large branch. A large summand bounds the sum below by order propagation.

Multiplication needs a separate zero analysis. Two finite cases give their ground product. A zero factor gives the zero branch using the corresponding left or right zero law. A nonzero finite factor is at least one; a large factor is also at least one because $1 \le n+1$. Hence a large factor paired with a nonzero factor bounds the product below by the appropriate guarded multiplication law. These arguments cover finite/large, large/finite and large/large pairs.

Apply the parent continuation to the resulting G and case proofs. The lemma `good_invariant_realized` in `FunctionalCases` instantiates Good at this proved constructor-closed motive. Finally choose $Z = \mathsf{Case}_n(d)$ and continuation $\lambda g\, h.h$. □

`DirectRFunctionalBounded.v` proves zero and successor closure; `DirectRFunctionalAddition.v` and `DirectRFunctionalMultiplication.v` prove the binary steps. Keeping both multiplication zero laws and the positive-factor premises is essential: an unconditional claim that a factor bounds its product is false when the other factor is zero.

The uniform rank accounting is:

Construction	Rank
Arithmetic carrier D	3
Finite selector domain $\delta_q(u)$	4
Individual observation certificates	at most 5
$E(d,e)$, $O(d,e)$ and $G(d)$	6
$\mathsf{Case}_n(d)$	8
$\mathsf{Inv}_n(d)$	10
$\mathsf{Good}(d)$	12

These ranks do not grow with the cutoff or axiom index. No minimality of the chosen ranks is claimed.

7 Realizing all five source schemes

The functional source language has $0, S, +, \cdot, =, \leq$. For external m, n, its schemes are the ground addition and multiplication equations, distinctness of unequal numerals, and

$$\text{R4: } \forall x.\ x \leq \bar{n} \to \bigvee_{i=0}^{n} x = \bar{i},$$
$$\text{R5: } \forall x.\ x \leq \bar{n} \vee \bar{n} \leq x.$$

`FunctionalSyntax.axiom` independently specifies these schemes. In particular, functional R4 is the displayed implication. The source definition does not import the relational presentation's biconditional.

`FunctionalAxioms.R4_realized` combines bounded cases with an assumed $O(d, \nu_n)$. Each finite branch is already a permitted equality; the large branch composes to $O(\nu_{n+1}, \nu_n)$ and is internally refuted. For `FunctionalAxioms.R5_realized`, finite branches give $O(d, \nu_n)$ by ground comparison and congruence. The large branch gives $O(\nu_n, d)$ by transitivity through ν_{n+1}.

Together with ground equations and separation, these give `all_axioms_realized` in `FunctionalAxioms` for the concrete axiom-type interface. The lemma `source_axiom_realized` in `FunctionalSourceAxioms` connects that result to the independently defined source syntax and its translation. Both are uniform in their indices, rather than a finite suite of examples.

8 First-order translation and preservation of deduction

`FirstOrder.term` and `FirstOrder.formula` are indexed by their number of free variables. Formulas have primitive equality, order, falsity, implication, conjunction, disjunction and both quantifiers. Terms translate compositionally to the four operations. Atoms use E, O; implication uses an arrow and the other connectives use the negative encodings. Quantifiers are relativized to admission:

$$(\forall x.A)^\dagger = \Pi x : D.\ \mathsf{Good}(x) \to A^\dagger,$$
$$(\exists x.A)^\dagger = \big(\Pi x : D.\ \mathsf{Good}(x) \to A^\dagger \to \bot\big) \to \bot.$$

The independent `FunctionalSyntax.derives` includes hypotheses, theory axioms, introduction and elimination for the primitive connectives, classical double-negation elimination, equality reflexivity and equality replacement. Its quantifier rules enforce the eigenvariable conditions through scope indices and explicit weakening of the assumption context. No constructor defines source derivability by target inhabitation.

Three layers make the induction on source derivations possible.

1. In `FunctionalTranslation`, `instantiate_translation` and the surrounding renaming/substitution lemmas connect source instantiation with actual target substitution.
2. In `FunctionalFormulaLogic`, `formula_formed` and `formula_stable` establish the formation and classical elimination obligations for every formula.
3. In `FunctionalEquality`, `formula_transport` proves equality transport between two typed, admitted environments that are pointwise E-related. Its quantified case extends both environments with the same admitted variable, preserving guards under binders.

This last condition matters: equality replacement is proved on admitted arguments. It does not require Good to be invariant under E on all raw D-objects. The proof does not obtain admission of one raw term merely from equality to another.

Theorem 8.1. *`FunctionalDeduction.preservation` translates an arbitrary derivation under assumptions into target inhabitation under any typed, admitted environment realizing those assumptions. Its closed specialization, `FunctionalDeduction.sentence_preservation`, states*

$$\mathsf{derives}([], A) \quad \Longrightarrow \quad \exists p.\ \vdash p : A^{\dagger}.$$

The proof follows the source derivation. Stable elimination handles conjunction projections, disjunction and existential elimination, and the classical rule. Quantifier introduction extends the target context with both an object and its admission proof; instantiation uses term admission and the substitution theorem. Equality replacement invokes the general formula-transport result. All final judgments are the original `Typing.typ` judgments.

9 Ground completeness and a precise nonfaithfulness witness

For a closed arithmetic term t, `FunctionalTerms.value` computes its external natural value. `FunctionalTerms.ground_evaluation` constructs $E(t^{\dagger}, \nu_{\mathsf{value}(t)})$ by structural induction, using constructor congruence and the ground equations. Ground truth then reduces to numeral equality or order; false cases use the finite separation proofs. `Theorem7.ground_correctness` packages this for `Theorem7.equality_atom` and `Theorem7.order_atom`.

For nonfaithfulness, take $\rho = \forall x.\ x \leq x$. `Theorem7.order_reflexivity_realized` proves $\rho^{\dagger}$: admission gives $G(d)$, which supplies the guard for each observation's reflexivity certificate.

`FunctionalCountermodel` independently proves that ρ is underivable. Its carrier is $\mathbb{N} \cup \{\omega\}$, represented by `option nat`. Successor fixes ω; addition and multiplication have their usual values on natural inputs and return zero whenever an input is ω. Order is ordinary order on naturals, every natural is below ω, and no ordered pair with left endpoint ω holds. Thus all five source schemes hold, whereas $\omega \not\leq \omega$.

The formal semantics uses stable negative interpretations for logical connectives. Decidability of atomic equality and order gives atomic stability; structural stability then validates the classical source rule constructively. `FunctionalCountermodel.soundness` proves soundness of every source derivation for this semantics, and `FunctionalCountermodel.order_reflexivity_underivable` obtains the contradiction at ω. Together these establish `Theorem7.nonfaithfulness` without a native classical axiom. This is a source countermodel, not a set-theoretic model of PTS-rk.

10 Effective proof terms and their computational limitations

An existential proof in Rocq's `Prop` is not itself an executable projection of its witness into `Type`. The development closes that gap using two shared modules, `DirectRDerivationCertificates.v` and `DirectRWitnessSearch.v`.

Finite tagged trees encode applications of exactly the original typing rules. Conversion certificates give finite forward beta paths to a common reduct; the original confluence characterization makes these certificates complete. `DerivationCertificates.derive_sound` proves that a successfully decoded certificate yields an original typing derivation, and `DerivationCertificates.derive_complete` proves that every original derivation has such a certificate.

`WitnessSearch.trees` enumerates finite sets with increasing bounds on tree depth and labels. Every certificate occurs at some stage. Searching a stage is decidable and returns a term only if its derived type is syntactically the requested type. Existence of an inhabitant, together with certificate completeness, proves that some stage succeeds. Constructive unbounded search over natural numbers therefore produces

$$\mathsf{witness}(A) : \mathsf{Realizes}(A) \to \{p : \mathsf{Term\ nat}\ 0 \mid\!\vdash p : A\}.$$

The Type-valued public interfaces in `Theorem7` are `effective_axiom`, `effective_source_axiom`, `effective_deduction`, `effective_ground_true` and `effective_ground_false`. Each returns a term paired with its original typing proof. In particular, `Theorem7.theorem5_target : Kernel.Theorem5Target` supplies a function selecting a typed witness for every concrete axiom instance. The historical record name does not identify a different source theory.

No choice axiom, typing-decidability assumption or fixed sufficient fuel bound is used. The procedure is intentionally exhaustive and has no practical complexity guarantee. For small explicit certificates the native checker is the appropriate computational interface. The older optional generator in `DirectRGenerated.v` and its named totality/soundness contracts in `DirectRObligations.v` are separate: completion of those contracts is not a premise of the effective interfaces above.

11 Source organization and reproduction

The source groups follow the mathematical dependencies:

- `DirectRCodes.v`, `DirectRFoundations.v` and `DirectRProofs.v` define the fixed annotated objects, certify their typing and prove ground laws.
- The `DirectRFinite` modules, together with the functional finite-law, evaluation and separation modules, implement and specialize the whole finite observation telescope.
- The functional bounded, addition, multiplication, cases and axioms modules prove the uniform invariant and all-index realization.
- The functional syntax, translation, formula-logic, source-axiom, equality and deduction modules connect independent source proofs to target inhabitants.
- `DirectRFunctionalCountermodel.v` and `DirectRTheorem7.v` supply nonfaithfulness and the final existential and effective interfaces. `DirectRFunctionalAudit.v` prints their types and transitive assumptions.

Use the repository's pinned environment: OCaml 4.14.2, Dune 3.23.1, Rocq core 9.2.0, Stdlib 9.1.0 and Equations 1.3.2+9.2. The setup is documented in `docs/DirectR-CI.md`. From the repository root:

```
dune build @theories/Interpretation_of_R/functional/audit
```

The following import exposes the public interfaces without importing the relational or set-model libraries:

```
From Interpretation_of_R.functional
  Require Import DirectRTheorem7.
Check Theorem7.theorem7.
Check Theorem7.theorem5_target.
Check Theorem7.effective_deduction.
Print Assumptions Theorem7.theorem7.
Print Assumptions Theorem7.theorem5_target.
Print Assumptions Theorem7.effective_deduction.
```

Use a fresh absolute Dune build directory when a new assumption report is required; cached success does not rerun printing commands. The full local check is `bash tools/check-local-ci.sh`, including actionlint, the staged developments and an isolated pristine upstream build.

The retained theories-layout verification of 14 October 2026 records a fresh complete build and public reports showing `theorem7`, `theorem5_target` and `effective_deduction` closed under the global context. Its commands and raw results are in the `theories-layout-2026-10-14` subdirectory of the handoff's verification archive. Consult `docs/direct-r-handoff/STATE.md` for the latest repository-wide verification. This documentation does not turn a cached build into a new kernel audit. Historical draft headers and manuscript status sentences describe their original checkpoints; current public declarations and live verification determine the theorem's present status.

12 Discussion

The functional construction keeps a fixed interface while allowing unboundedly large finite proofs. The expression carrier exposes all four constructors; the observation telescope supplies just enough algebra to reason about them. Finite selectors provide genuine target proofs of separation, and the bounded invariant transports the positive evidence needed to use those algebraic laws through negative case analysis.

This also explains the interpretation's boundaries. It does not claim that every raw carrier term is admitted, that observational equality coincides with beta conversion, or that arbitrary source theorems can be recovered from target proofs. Uniformly bounded ranks concern the encoding, not the size or running time of the externally constructed witnesses. The checked result establishes the full stated interpretation and its constructive witness interface; rank optimization and an efficient proof compiler are separate possible improvements.